Mensajería con Kafka y RabbitMQ + Spring Batch

Repaso técnico — Java 21 · Spring Boot · Arquitectura Hexagonal

1. RabbitMQ vs Kafka — Diferencias clave

Ambos resuelven comunicación asíncrona entre servicios, pero con filosofías distintas. Elegir mal complica el sistema innecesariamente.

	RabbitMQ	Kafka
Modelo	Broker con push a consumidor	Log distribuido, consumidor hace pull
Mensaje procesado	Desaparece al consumir	Persiste (días / indefinido)
Replay	No	Sí, el consumidor elige su offset
Múltiples consumidores	Cada mensaje lo lee uno solo	Grupos independientes leen el mismo evento
Orden	Por cola, no garantizado entre consumers	Garantizado dentro de una partición
Terminología	Exchange → Queue → Consumer	Producer → Topic/Partición → Consumer Group
Patrón natural	Tarea: "procesa esto"	Hecho: "esto ocurrió"
Complejidad operacional	Menor	Mayor (brokers, réplicas, Schema Registry…)

Regla práctica: el volumen alto por sí solo no justifica Kafka sobre RabbitMQ. RabbitMQ aguanta decenas de miles de mensajes por segundo. Kafka se justifica cuando múltiples servicios independientes reaccionan al mismo evento, necesitas replay, o requieres una traza inmutable.

Cuándo elegir cada uno

Situación	Elige
Una tarea que procesa un solo servicio	RabbitMQ
Reintentos con backoff, TTL, DLQ	RabbitMQ
Varios servicios reaccionan al mismo evento	Kafka
Necesitas releer eventos del pasado (replay)	Kafka
Auditoría legal / event sourcing	Kafka
Llamada externa lenta, volumen bajo	RabbitMQ (o incluso @Async)

2. RabbitMQ — Flujo con ejemplo de denuncias

Escenario: una denuncia entra por REST, se valida internamente y se delega a un servicio externo lento. Solo ese servicio la procesa. RabbitMQ es la elección natural.

Arquitectura de componentes

```
[POST /denuncia] | ▼ [Controller] → [Use Case: validar + transformar] | ▼ [RabbitPedidoPublisher] ⟵ puerto de salida (dominio) | publica a ▼ [Exchange: denuncias.exchange] ⟶routing key⟶ [Queue: denuncias] | [RabbitDenunciaConsumer] | ▼ [Use Case: procesar] | si falla → [DLQ: denuncias.dead]
```

Configuración de colas

```java
@Configuration
public class RabbitDenunciaConfig {

    @Bean
    public Queue denunciasQueue() {
        return QueueBuilder.durable("denuncias")
                .withArgument("x-dead-letter-exchange", "denuncias.dlx")
                .build();
    }

    @Bean
    public TopicExchange denunciasExchange() {
        return new TopicExchange("denuncias.exchange");
    }

    @Bean
    public Binding binding(Queue denunciasQueue, TopicExchange denunciasExchange) {
        return BindingBuilder.bind(denunciasQueue)
                .to(denunciasExchange).with("denuncias.#");
    }
}
```

Productor — adaptador de salida

```java
@Component
public class RabbitDenunciaPublisher implements DenunciaEventPort {

    private final RabbitTemplate rabbitTemplate;

    @Override
    public void publicar(DenunciaCreadaEvent event) {
        rabbitTemplate.convertAndSend(
            "denuncias.exchange", "denuncias.creada", event
        );
    }
}
```

Consumidor — adaptador de entrada

```java
@Component
public class RabbitDenunciaConsumer {

    private final ProcesarDenunciaUseCase useCase;

    @RabbitListener(queues = "denuncias")
    public void onMessage(DenunciaCreadaEvent event, Channel channel,
                          @Header(AmqpHeaders.DELIVERY_TAG) long tag) throws Exception {
        try {
            useCase.procesar(event);
            channel.basicAck(tag, false);
        } catch (Exception e) {
            channel.basicNack(tag, false, false); // false = no requeue → va al DLX
        }
    }
}
```

Componentes clave de RabbitMQ

Componente	Rol
Exchange	Decide a qué cola va el mensaje. Tipos: direct, topic, fanout, headers
Binding	Enlace entre exchange y cola con una routing key
DLX / DLQ	Mensajes rechazados o expirados van aquí para inspección
TTL	Tiempo de vida del mensaje. Base del patrón de reintento con backoff
Prefetch / QoS	Cuántos mensajes sin confirmar puede acumular un consumer
Publisher Confirms	El broker confirma al productor que recibió el mensaje
Ack / Nack	El consumer confirma o rechaza cada mensaje explícitamente

Reintento con TTL (patrón Dead Letter)

```
[Cola principal: denuncias] | falla consumer ▼ [Cola retry: denuncias.retry] ← mensaje espera N ms (TTL) | TTL expira → "muere" ▼ [DLX] → reencola en [denuncias] | | si falla N veces ▼ [DLQ: denuncias.dead] ← inspección manual / alerta
```

Esto consigue backoff sin código complejo: el mensaje espera en la cola de retry antes de volver a intentarse. El tiempo de espera lo controla el TTL de esa cola.

3. Kafka — Flujo con ejemplo de denuncias

Escenario: una denuncia se crea y varios servicios independientes reaccionan: moderación evalúa su validez, notificaciones avisa al afectado, y auditoría la registra. Kafka es la elección correcta porque el mismo evento lo consumen múltiples sistemas.

Arquitectura de componentes

```
[POST /denuncia] | ▼ [Controller] → [Use Case: validar + transformar] | ▼ [KafkaDenunciaPublisher] ⟵ puerto de salida | produce → topic: denuncias-events (clave: denunciaId) ▼ [Topic Kafka: denuncias-events] ├─ Partición 0 ├─ Partición 1 └─ Partición 2 | ├──▶ [Consumer Group: moderacion-service] → evalúa validez ├──▶ [Consumer Group: notificaciones-service] → avisa al afectado └──▶ [Consumer Group: auditoria-service] → traza legal inmutable
```

Clave de particionado: usar `denunciaId` como clave garantiza que todos los eventos del mismo expediente van a la misma partición, preservando el orden.

Productor — adaptador de salida

```java
@Component
public class KafkaDenunciaPublisher implements DenunciaEventPort {

    private final KafkaTemplate<String, DenunciaCreadaEvent> kafkaTemplate;

    @Override
    public void publicar(DenunciaCreadaEvent event) {
        kafkaTemplate.send("denuncias-events", event.denunciaId(), event);
    }
}
```

Consumidor — adaptador de entrada

```java
@Component
public class KafkaDenunciaConsumer {

    private final ModerarDenunciaUseCase useCase;

    @KafkaListener(topics = "denuncias-events", groupId = "moderacion-service")
    public void onMessage(DenunciaCreadaEvent event, Acknowledgment ack) {
        useCase.moderar(event);
        ack.acknowledge(); // commit manual del offset
    }
}
```

Componentes clave de Kafka

Componente	Rol

Topic	Canal de mensajes. Equivalente conceptual a la "cola", pero persistente
Partición	Subdivisión del topic. Más particiones = más paralelismo
Offset	Posición del consumer en el log. Permite replay desde cualquier punto
Consumer Group	Grupo que comparte la carga de un topic. Cada partición la lee un solo miembro
Broker	Nodo del clúster que almacena particiones
Replication Factor	Copias de cada partición en distintos brokers. Mínimo 3 en producción
ISR	Réplicas al día con el líder. Si el líder cae, una ISR toma el relevo
Schema Registry	Valida que productor y consumer usen el mismo esquema (Avro, Protobuf)
Retention	Tiempo que Kafka guarda los mensajes. Por defecto 7 días
@RetryableTopic	Spring Kafka: crea topics de reintento y DLT automáticamente

4. Arquitectura basada en eventos con Kafka

En la arquitectura orientada a eventos, **el evento es la fuente de verdad**, no el estado actual de la base de datos. El sistema reacciona a hechos que ocurrieron, no a comandos directos.

Principios

- **Eventos inmutables:** una vez publicado, un evento no se modifica. "DenunciaCreada", "DenunciaModerada", "DenunciaArchivada".
- **Desacoplamiento total:** el servicio que publica no sabe quién consume ni cuántos.
- **Event sourcing:** el estado se reconstruye reproduciendo todos los eventos desde el origen. La BD es un proyección derivada, no la verdad.

Patrón Outbox — evitar doble escritura

```
1. El use case escribe en BD + tabla outbox en la MISMA transacción 2. Un proceso lee la outbox y publica en Kafka 3. Si Kafka falla, la outbox conserva el evento pendiente → Garantía de entrega sin perder consistencia
```

> Sin el patrón Outbox, una escritura en BD exitosa + fallo al publicar en Kafka produce un evento perdido silenciosamente. En sistemas de auditoría legal esto es crítico.

Kafka en arquitectura hexagonal

El dominio nunca conoce Kafka. Define **puertos** (`DenunciaEventPort`) y los **adaptadores de infraestructura** los implementan. Los consumidores son adaptadores de entrada que traducen eventos externos a llamadas al caso de uso del dominio.

5. Spring Batch — Repaso

Framework para procesar grandes volúmenes de datos por lotes: leer de una fuente, transformar, escribir en un destino. Diseñado para ser robusto: reintentos, skip de registros erróneos, reinicio desde donde falló, trazabilidad completa.

Arquitectura

```
[Job] └── [Step 1: chunk-oriented] ├── ItemReader → lee registros (CSV, BD, API ...) ├── ItemProcessor → transforma cada registro (null = skip) └── ItemWriter → escribe resultados (BD, fichero ...) └── [Step 2: tasklet] └── lógica arbitraria (mover fichero, enviar email, limpiar tabla ...)
```

Chunk: tamaño del lote procesado en cada transacción. Con `chunkSize=100`: lee 100, procesa 100, escribe 100 en una sola transacción. Si falla, el rollback afecta solo a ese chunk.

JobRepository: Spring Batch guarda en BD el estado de cada ejecución. Imprescindible para reinicio y trazabilidad.

Configuración completa — importar denuncias desde CSV

```java
@Configuration
public class DenunciaBatchConfig {

    @Bean
    public FlatFileItemReader<DenunciaRaw> reader() {
        return new FlatFileItemReaderBuilder<DenunciaRaw>()
                .name("denunciaReader")
                .resource(new ClassPathResource("denuncias.csv"))
                .delimited().names("id", "descripcion", "fecha")
                .targetType(DenunciaRaw.class)
                .build();
    }

    @Bean
    public ItemProcessor<DenunciaRaw, Denuncia> processor() {
        return raw -> {
            if (raw.getDescripcion().isBlank()) return null; // null = skip silencioso
            return new Denuncia(raw.getId(), raw.getDescripcion().trim(),
                                LocalDate.parse(raw.getFecha()));
        };
    }

    @Bean
    public JdbcBatchItemWriter<Denuncia> writer(DataSource dataSource) {
        return new JdbcBatchItemWriterBuilder<Denuncia>()
                .dataSource(dataSource)
                .sql("INSERT INTO denuncia (id, descripcion, fecha) VALUES (:id, :descripcion, :fecha)")
                .beanMapped().build();
    }

    @Bean
    public Step procesarStep(JobRepository jobRepository, PlatformTransactionManager txManager,
                             FlatFileItemReader<DenunciaRaw> reader,
```

```
                            ItemProcessor<DenunciaRaw, Denuncia> processor,
                            JdbcBatchItemWriter<Denuncia> writer) {
        return new StepBuilder("procesarDenunciasStep", jobRepository)
                .<DenunciaRaw, Denuncia>chunk(100, txManager)
                .reader(reader).processor(processor).writer(writer)
                .faultTolerant()
                    .skip(CsvParseException.class).skipLimit(10)
                    .retry(TransientDataAccessException.class).retryLimit(3)
                .build();
    }

    @Bean
    public Job importarDenunciasJob(JobRepository jobRepository, Step procesarStep) {
        return new JobBuilder("importarDenunciasJob", jobRepository)
                .listener(new DenunciaJobListener())
                .start(procesarStep)
                .build();
    }
}
```

Arrancar el Job

```
@Service
public class BatchLauncherService {

    private final JobLauncher jobLauncher;
    private final Job importarDenunciasJob;

    public void lanzar() throws Exception {
        JobParameters params = new JobParametersBuilder()
                .addLocalDateTime("timestamp", LocalDateTime.now()) // para poder relanzar
                .toJobParameters();
        jobLauncher.run(importarDenunciasJob, params);
    }
}
```

> Spring Batch no permite relanzar un Job con los mismos parámetros si ya completó con éxito. Por eso se añade un `timestamp` u otro valor único como parámetro.

Hooks — JobExecutionListener

```
@Component
public class DenunciaJobListener implements JobExecutionListener {

    @Override
    public void beforeJob(JobExecution jobExecution) {
        log.info("Iniciando job: {}", jobExecution.getJobInstance().getJobName());
    }

    @Override
    public void afterJob(JobExecution jobExecution) {
        if (jobExecution.getStatus() == BatchStatus.FAILED) {
            // notificar, limpiar, alertar ...
```

```
        }
    }
}
```

Formas de lanzar un Job

Forma	Cuándo
Endpoint REST (`@PostMapping`)	Lanzado manualmente o por otro sistema
`@Scheduled(cron = "...")`	Procesamiento nocturno o periódico
Al recibir un evento de Kafka/Rabbit	El batch se dispara al llegar un fichero o señal externa
Al arrancar la app (`job.enabled: true`)	Solo para procesos de un solo disparo o entornos de prueba

Componentes extra a conocer

Componente	Para qué
`JobExecutionListener`	Hooks antes/después del job completo
`StepExecutionListener`	Hooks antes/después de cada step
`ExecutionContext`	Compartir datos entre steps del mismo job
`Partitioner`	Dividir un step en subtareas paralelas
`AsyncItemProcessor`	Procesar registros en paralelo con un thread pool
`JobParameters`	Parámetros de entrada: fichero a procesar, fecha, etc.

Flujo mental para crear un batch nuevo

```
1. ¿Cuál es la fuente? → elige el Reader (CSV, BD, API, fichero fijo) 2. ¿Qué transformación? → implementa el Processor (o déjalo si no hay) 3. ¿Dónde va el resultado? → elige el Writer (BD, fichero, otro servicio) 4. ¿Cuánto chunkSize? → 100–500 como punto de partida 5. ¿Qué pasa si falla un registro? → configura skip / retry 6. ¿Cómo se lanza? → REST, @Scheduled, evento, o al arrancar
```

Repaso generado en sesión de estudio · Java 21 · Spring Boot · Arquitectura Hexagonal